

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I _____ (V. Asha Sarthiwika / 192311066) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:V. Asha Sarthiwika

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a)** Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b)** Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c)** Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a)** Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, – 1 each step, – 5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b)** Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c)** Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Annexure A – Integrated Experiments

Experiment 1: Decision Tree Classification

Objective

To implement a Decision Tree classifier using Python and evaluate its performance using accuracy, confusion matrix, and classification report.

(a) Dataset Loading and Pre-processing

For this experiment, the Iris dataset can be used. It contains 150 samples with four input features:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

There are three classes:

- Setosa
- Versicolor
- Virginica

The dataset has no missing values, so no missing-value treatment is required. Since all input features are numerical, encoding is also not required.

Python code:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
import pandas as pd

# Load dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(
    iris.data,
    columns=iris.feature_names
)

df["target"] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())
```

```

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Split features and target
X = df.drop("target", axis=1)
y = df["target"]

# Training and testing split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

```

Sample First 5 Rows:

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

(b) Build the Decision Tree

The Decision Tree can use **Gini Index** as the splitting criterion.

Python code:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text, plot_tree
import matplotlib.pyplot as plt
```

```
# Create Decision Tree
```

```
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
```

```
# Train model
```

```
model.fit(X_train, y_train)
```

```
# Print tree structure
```

```
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)
```

```
print("Decision Tree Structure:")
```

```
print(tree_rules)
```

```
# Root node
```

```
root_feature = X.columns[model.tree_.feature[0]]
print("\nRoot Node:", root_feature)
```

```
# Plot tree
```

```
plt.figure(figsize=(14, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)
```

```
plt.show()
```

Root Node

For the Iris dataset, the Decision Tree commonly selects:

Petal Length (cm)

as the root node when using the specified training split and Gini criterion.

The root is selected because it provides the **largest reduction in impurity**, producing the most effective separation of the three classes at the first split.

(c) Model Evaluation

Python code:

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
```

```
target_names=iris.target_names
)
)
```

Evaluation

The Decision Tree generally provides **high accuracy on the Iris dataset**, often around 90–100%, depending on the train-test split and tree configuration.

The confusion matrix shows how many samples from each class were correctly and incorrectly classified.

Misclassified Instances

The exact misclassified samples depend on the random split and tree configuration. They can be identified using:

Python Code:

```
misclassified = X_test[y_test != y_pred].copy()

misclassified["Actual"] = y_test[y_test != y_pred]
misclassified["Predicted"] = y_pred[y_test != y_pred]

print("\nMisclassified Instances:")
print(misclassified.head(2))
```

This prints at least two misclassified instances when such instances exist.

Performance Comment

The Decision Tree performs well because the Iris classes, especially **Setosa**, are easily separated using petal measurements. Some overlap may occur between Versicolor and Virginica, which can result in misclassification. The model is simple to interpret because its decisions can be represented as a tree structure.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement Q-Learning in a 4×4 Grid World and learn an optimal policy through interaction with the environment.

(a) Environment Definition

A **4×4 Grid World** contains 16 states.

The states can be represented as:

```
0  1  2  3
4  5  6  7
8  9 10 11
12 13 14 15
```

Assume:

- **State 15** = Goal

- **State 5** = Obstacle
- Actions = Up, Down, Left, Right

Reward Structure

Situation	Reward
Reach Goal	+10
Normal movement	-1
Move into obstacle	-5

Hyperparameters

Learning rate:

Discount factor:

Exploration probability:

Q-Learning Formula

where:

- s = current state
- a = current action
- r = reward
- s' = next state
- α = learning rate
- γ = discount factor

Python Implementation

```
import numpy as np
```

```
epsilon = 0.1
```

```
# Initialize Q-table
```

```
Q = np.zeros((num_states, num_actions))
```

```
# Function to move
```

```
def step(state, action):
```

```
    row = state // COLS
```

```
    col = state % COLS
```

```
    new_row = row
```

```
    new_col = col
```

```
if action == 0:    # Up
    new_row -= 1
elif action == 1:  # Down
    new_row += 1
elif action == 2:  # Left
    new_col -= 1
elif action == 3:  # Right
    new_col += 1

# Boundary check
if new_row < 0 or new_row >= ROWS or new_col < 0 or new_col >= COLS:
    return state, -1

next_state = new_row * COLS + new_col

# Obstacle
if next_state == obstacle:
    return state, -5

# Goal
if next_state == goal:
    return next_state, 10

return next_state, -1
```

Training

episodes = 100

rewards = []

for episode in range(1, episodes + 1):

state = 0

total_reward = 0

```

for step_count in range(100):

    # Epsilon-greedy action selection
    if random.random() < epsilon:
        action = random.randint(0, num_actions - 1)
    else:
        action = np.argmax(Q[state])

    next_state, reward = step(state, action)

    # Q-learning update
    Q[state, action] = Q[state, action] + alpha * (
        reward +
        gamma * np.max(Q[next_state]) -
        Q[state, action]
    )

    state = next_state
    total_reward += reward

    if state == goal:
        break

rewards.append(total_reward)

# Display Q-values at selected episodes
if episode in [1, 50, 100]:
    print("\nEpisode:", episode)
    print("State 0 Q-values:", Q[0])
    print("State 10 Q-values:", Q[10])

# Display final Q-table
print("\nFinal Q-Table:")
print(Q)

```

(b) Q-Table Evolution

The Q-table initially contains only zeros:

$$Q = 0$$

Episode 1

The agent has almost no knowledge about the environment. Therefore, most Q-values remain close to zero.

Example format:

State 0: [0.0, -0.1, -0.1, 0.0]

State 10: [0.0, 0.0, 0.0, 0.0]

Episode 50

After repeated interaction, the agent begins learning which actions lead toward the goal.

Example:

State 0: [negative value, negative value, negative value, positive value]

State 10: [positive value, positive value, negative value, positive value]

The exact values depend on the random exploration sequence.

Episode 100

The Q-values become more stable. Actions leading toward the goal generally obtain higher values than actions leading away from the goal.

The program automatically records the actual Q-values for Episodes **1, 50, and 100**.

(c) Final Learned Policy

The optimal action is selected using:

Python code:

```
policy = []
```

```
for state in range(num_states):
```

```
    if state == goal:
```

```
        policy.append("Goal")
```

```
    elif state == obstacle:
```

```
        policy.append("Obstacle")
```

```
    else:
```

```
        best_action = np.argmax(Q[state])
```

```
        policy.append(actions[best_action])
```

```
print("\nFinal Learned Policy:")
```

```
for row in range(ROWS):
```

```
    print(policy[row * COLS:(row + 1) * COLS])
```

A typical learned policy will direct the agent generally toward the goal while avoiding the obstacle.

For example:

```
Right  Right  Right  Down
```

```
Right  Obstacle Right  Down
```

```
Right  Right  Right  Down
```

```
Right  Right  Right  Goal
```

The exact policy can differ because Q-Learning involves random exploration.

Cumulative Reward Plot

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(rewards)
```

```
plt.xlabel("Episode")
```

```
plt.ylabel("Cumulative Reward")
```

```
plt.title("Q-Learning: Cumulative Reward per Episode")
```

```
plt.grid()
```

```
plt.show()
```

Convergence Behaviour

During the initial episodes, the agent explores the environment and may receive low or negative rewards because it does not yet know the optimal path.

As training progresses:

1. The Q-values are updated.
2. The agent learns which actions provide better rewards.
3. The agent increasingly chooses actions leading toward the goal.
4. The cumulative reward generally improves.
5. Eventually, the learned policy becomes more stable.

Therefore, the increasing and eventually stabilizing reward curve indicates that the Q-Learning agent is **learning and approaching a stable policy**.